

Guia de comandos básicos

Estos comandos son esenciales porque la terminal es el motor central de Linux. A diferencia de otros sistemas operativos, la línea de comandos en Linux no es un accesorio, sino la forma más directa y potente de controlar tu computadora.

1. El Comando `ls` y sus Modificadores

Concepto Principal: La salida del comando `ls` lista el contenido de los directorios y permite distinguir visualmente los objetos mediante colores. Por ejemplo, en verde se ven los ejecutables o scripts, en celeste los enlaces simbólicos, en rojo los comprimidos, y en letras rojas con fondo negro los enlaces rotos. Los archivos que empiezan con un punto (.) están ocultos y suelen usarse para configuraciones.

Comandos Clave:

- ~ `ls -a`: Muestra todos los archivos, incluyendo los ocultos.
- ~ `ls -l`: Muestra información detallada (permisos, enlaces, dueño, grupo, tamaño en bytes, fecha de modificación).
- ~ `ls -lh`: Al agregar la `h` (human-readable), los tamaños se muestran en unidades más amigables (KB, MB, etc.).
- ~ `ls -R`: Lista el contenido del directorio y sus subdirectorios de forma recursiva.

Ejemplos Prácticos:

- ~ `ls -lh adm.txt`: Muestra los detalles de ese archivo puntual, con su tamaño en un formato legible (ej. 3,9K).
PDF
- ~ `ls -lR pruebas`: Devuelve un listado detallado de la carpeta "pruebas" y de todas las carpetas en su interior.

Perspectiva de Ciberseguridad: Revisar archivos ocultos con `ls -a` es el paso cero en auditorías e investigaciones. Los atacantes frecuentemente ocultan sus herramientas maliciosas, scripts o carpetas de exfiltración de datos simplemente anteponiendo un punto a sus nombres.

2. Gestión de Espacio en Disco (`du` y `df`)

Concepto Principal: Ambas herramientas supervisan el almacenamiento, pero con enfoques distintos. `du` (Disk Usage) evalúa qué espacio físico ocupan uno o varios archivos/directorios puntuales. `df` (Disk Free) permite ver el panorama general: cuánto espacio libre y usado hay por cada partición o volumen lógico montado en el sistema.

Comandos Clave:

~ `du -ch`: Muestra el uso de disco (`h` para legible) y agrega una línea con el total (`c`).

~ `df -h`: Despliega el estado de los sistemas de archivos, mostrando tamaño total, usado, disponible y punto de montaje de manera legible.

Ejemplos Prácticos:

~ `du -ch pruebas/*`: Evalúa el tamaño de cada archivo dentro de la carpeta "pruebas" y da la suma total.

~ `df -h`: Monitorea el espacio de particiones críticas como `/`, `/var`, `/home` y `/tmp`.

Perspectiva de Ciberseguridad: Un vector de ataque local común de denegación de servicio (DoS) consiste en llenar deliberadamente una partición (como `/var/log` inundando los registros). Monitorear periódicamente los porcentajes de uso con `df` previene la caída de servicios críticos.

3.El Historial de la Terminal (**history**)

Concepto Principal: Este comando expone y enumera cuáles fueron las instrucciones ingresadas por el usuario leyendo el archivo oculto **.bash_history** ubicado en su directorio **home**. Permite visualizar la lista numerada en orden cronológico.

Ejemplos Prácticos:

~ **history**: Devuelve la lista de los últimos comandos ejecutados.

~ **!382**: Vuelve a ejecutar inmediatamente el comando asociado a ese número sin tener que tipear de nuevo.

Perspectiva de Ciberseguridad: El archivo **.bash_history** es una mina de oro en el análisis forense. Si un atacante logra comprometer una cuenta, el historial revelará qué archivos modificó o qué privilegios intentó escalar. Proteger los permisos de este archivo es vital.

4. Manipulación de Directorios y Archivos

Concepto Principal: Herramientas de gestión diaria para estructurar el file system, copiar datos, moverlos o eliminarlos.

Comandos Clave y Ejemplos Prácticos:

- ~ **mkdir** (Crear): **mkdir -p dir1/dir2/dir3** crea una estructura completa de carpetas anidadas en un solo paso.
- ~ **rmdir** (Borrar carpeta): Elimina directorios, pero solo si están completamente vacíos.
- ~ **cp** (Copiar): **cp -a pruebas/holamundo.txt destino/** hace una copia exacta. La opción **-a** es importante porque preserva permisos, enlaces y copia subdirectorios tal cual.
- ~ **mv** (Mover / Renombrar): **mv copia.txt nueva.txt** renombra el archivo. Sobrescribe el destino si este ya existe.
- ~ **touch** (Crear / Actualizar): **touch archivo.txt** crea un archivo vacío si no existe, o actualiza su fecha de última modificación si ya existe.
- ~ **rm** (Eliminar): Borra archivos sin posibilidad de recuperarlos (no hay papelera). **rm -ri carpeta** borra de forma recursiva (**-r**) pero pidiendo confirmación interactiva (**-i**) por cada archivo.

Perspectiva de Ciberseguridad:

Borrado: La opción **-i** en **rm** previene accidentes catastróficos que puedan comprometer la disponibilidad del servidor.

RespalDOS: Copiar configuraciones con **cp -a** es seguro ya que no altera la estructura de permisos y propietarios original (Principio de Menor Privilegio).

Timestomping: La propiedad del comando **touch** para alterar la fecha de los archivos es una técnica que utilizan los atacantes para ocultar malware, haciéndolo pasar por un archivo legítimo y antiguo del sistema operativo.

5. Enlaces (ln)

Concepto Principal: Permite crear referencias (links) a otros recursos.

Hard links (Duros): Enlazan directamente al mismo inodo y bloques del archivo. No pueden apuntar a carpetas ni salir de la partición actual.

Soft links (Simbólicos): Representan un enlace a otro inodo (como un acceso directo). Son extremadamente útiles para apuntar a versiones actualizadas de un programa (ej. enlazar Y hacia Y-version2) sin necesidad de recompilar dependencias.

Ejemplo Práctico:

~ `ln -s archivo_original enlace_simbolico`: Crea un Soft Link.

Perspectiva de Ciberseguridad: Los enlaces simbólicos rotos (resaltados por `ls` en la terminal) pueden ser un indicador temprano de que dependencias críticas del sistema fueron eliminadas o comprometidas.

6. Ciclo de Energía (Apagar y Reiniciar)

Concepto Principal: Aunque los servidores Linux pueden funcionar años sin detenerse, a veces requieren mantenimiento, reemplazo de hardware o simulacros. Estos comandos permiten manejar el ciclo de energía de manera abrupta o controlada.

Comandos Clave y Ejemplos Prácticos:

~ Apagar: `init 0, poweroff, halt, shutdown -h o systemctl poweroff`.

~ Reiniciar: `init 6, reboot, shutdown -r o systemctl reboot`.

~ Apagado/Reinicio Programado: `shutdown -r 14:00 'Reinicio por mantenimiento'` reinicia el servidor a las 14hs y envía un mensaje a las terminales.

~ Cancelar Apagado: `shutdown -c`.

Perspectiva de Ciberseguridad: El comando `shutdown` es la herramienta preferida frente a comandos abruptos como `halt`. Permite enviar advertencias e impide nuevos logins mientras los servicios se detienen limpiamente y los sistemas de archivos se desmontan con cuidado, evitando la corrupción de bases de datos o la pérdida de registros de auditoría en memoria.

Ejercicio práctico

Importante: Este laboratorio se realiza con un usuario sin permisos de superusuario (que no sea root).

1. Crear un entorno de práctica

```
mkdir ~/lab1
cd ~/lab1
mkdir carpeta1 carpeta2
touch archivo1.txt
cp archivo1.txt carpeta1/
```

```
luck4@UTN-Lucka:~$ mkdir ~/lab1
luck4@UTN-Lucka:~$ cd ~/lab1
luck4@UTN-Lucka:~/lab1$ mkdir carpeta1 carpeta2
luck4@UTN-Lucka:~/lab1$ touch archivo1.txt
luck4@UTN-Lucka:~/lab1$ cp archivo1.txt carpeta1/
luck4@UTN-Lucka:~/lab1$
```

Por que usamos `~/` ? Por que `~/` es un atajo predecible al home y que `cd carpeta1` depende enteramente de dónde estés parado.

`~ cd carpeta1`: Ruta relativa al directorio actual.

`~ cd /home/usuario/lab4/carpeta1`: Ruta absoluta.

`~ cd ~/lab4/carpeta1`: Ruta relativa al home (un híbrido cómodo).

Luego para volver a la carpeta anterior se puede usar `cd -` o `cd ..` para volver a la carpeta padre (un nivel arriba). Para el home con `cd` bastará.

Pero si quisiéramos crear un directorio y subdirectorios?

```
luck4@UTN-Lucka:~$ cd ~/lab1/
luck4@UTN-Lucka:~/lab1$ mkdir directorio1/subdirectorio/subdelsb
mkdir: cannot create directory 'directorio1/subdirectorio/subdelsb': No such file or directory
luck4@UTN-Lucka:~/lab1$
luck4@UTN-Lucka:~/lab1$
luck4@UTN-Lucka:~/lab1$ mkdir -p directorio1/subdirectorio/subdelsb
luck4@UTN-Lucka:~/lab1$ ls
archivo1.txt  carpeta1  carpeta2  directorio1
```

Como vemos en el primer intento el comando falló y la terminal devolvió un mensaje de error.

Para crear la estructura completa de una sola vez, debemos utilizar la opción `-p` (parents / padres). Esta bandera le instruye a `mkdir` que cree cualquier directorio padre que falte silenciosamente antes de crear el directorio final.

2. Rutas relativas y absolutas

```
luck4@UTN-Lucka:~/lab1/carpeta1$ touch dentro.txt
luck4@UTN-Lucka:~/lab1/carpeta1$ cp dentro.txt ../carpeta2/
luck4@UTN-Lucka:~/lab1/carpeta1$ cp /home/luck4/lab1/carpeta1/dentro.txt /home/luck4/lab1/carpeta2/
```

a. Por qué se utiliza ../carpeta2/?

~ un punto solo: Representa el Directorio Actual de Trabajo. Es el "aquí".

~ dos puntos: Representa el Directorio Padre, el directorio que está exactamente un nivel por encima en la jerarquía del sistema de archivos. Si estás en /home/usuario/lab, .. es /home/usuario.

b. Cambia algo usar ../carpeta2 y cp

/home/\$USER/lab4/carpeta1/dentro.txt /home/\$USER/lab4/carpeta2/ ?

~ Comando 1: Ruta Relativa (../carpeta2) Este comando le está diciendo al sistema: "Copia el archivo dentro.txt a un directorio llamado carpeta2, que se encuentra un nivel por encima de donde estoy ahora".

~ Comando 2: Ruta Absoluta (/home/\$USER/lab4/carpeta2/) Este comando le está diciendo al sistema: "Copia el archivo dentro.txt exactamente a esta ubicación física, empezando desde la raíz (/) del sistema de archivos, pasando por el home de mi usuario (\$USER), hasta llegar a lab4/carpeta2/".

3. Exploración del sistema de archivos

```
luck4@UTN-Lucka:~$ pwd
/home/luck4
luck4@UTN-Lucka:~$ cd /
luck4@UTN-Lucka:/$ ls
bin  dev  home  initrd.img.old  lib64      media  opt   root  sbin  sys  usr  vmlinuz
boot etc  initrd.img  lib          lost+found  mnt    proc  run   srv   tmp  var  vmlinuz.old
luck4@UTN-Lucka:/$
```

/Home: Contiene los datos personales y las configuraciones específicas de cada usuario

/etc: Contiene todos los archivos de configuración globales del sistema y de los servicios.

/usr/bin: Contiene la inmensa mayoría de los binarios (ejecutables) y comandos estándar preinstalados o instalados por el gestor de paquetes (como apt), que están disponibles para todos los usuarios (comandos como ls, cat, grep).

/tmp: Directorio de archivos temporales. Cualquier programa o usuario puede escribir acá para guardar datos transitorios.

/var/log: Contiene los registros (logs). Aunque journalctl (de systemd) centraliza muchos logs hoy en día, muchísimos servicios tradicionales (como Apache, Nginx, o el gestor de paquetes apt) siguen escribiendo archivos de texto plano directamente acá.

Por último, un cheat sheet para tener a mano.

